

DOCUMENTO TÉCNICO DE DISEÑO DE BASE DE DATOS

Arquitectura Híbrida (PostgreSQL - MongoDB) para Ecommify

Autores: Ricardo Castro, [Nombres Adicionales del Grupo 9]

Grupo: 9

Fecha: Mayo 2026

A. Resumen Ejecutivo (Abstract)

Este documento detalla el diseño conceptual y lógico de la arquitectura de base de datos híbrida para Ecommify, una plataforma de e-commerce multivendedor, utilizando el dataset de Olist. Ante la necesidad de equilibrar la consistencia transaccional y la disponibilidad analítica, se adoptó un enfoque polígloa. El módulo transaccional (pedidos, pagos, inventario) se modeló en PostgreSQL bajo la tercera forma normal (3FN), garantizando el cumplimiento ACID. Se implementaron tipos avanzados (JSONB , ARRAY) para datos semiestructurados de productos, y extensiones espaciales (PostGIS) y de texto borroso (pg_trgm) para optimizar logística y búsquedas. Para mitigar cuellos de botella por crecimiento, se aplicó particionamiento por rangos en la tabla de pedidos. Paralelamente, el catálogo extendido y el registro de eventos se diseñaron en MongoDB (NoSQL) priorizando alta disponibilidad y escalabilidad horizontal de lectura. Esta combinación asegura resiliencia operativa y rendimiento óptimo en cargas de trabajo mixtas (OLTP/OLAP), alineándose estrictamente con los teoremas arquitectónicos modernos.

(Palabras: ~165)

B. Análisis de Requisitos

Contexto de Ecommify

Ecommify es una plataforma de comercio electrónico multivendedor que requiere un ecosistema de datos capaz de soportar tanto el flujo crítico del negocio (compras, facturación) como la experiencia del usuario (búsquedas rápidas, catálogo dinámico). La arquitectura debe balancear la integridad financiera con la flexibilidad necesaria para un catálogo de productos altamente variable.

Requisitos Funcionales y No Funcionales

Funcionales:

- Garantizar que una compra descuente el stock y registre el pago sin anomalías (Atomicidad y Consistencia).
- Soportar búsquedas de productos con tolerancia a errores ortográficos.
- Calcular costos de envío basados en distancias geoespaciales reales entre vendedor y comprador.
- Almacenar especificaciones de productos que varían según su categoría sin requerir alteraciones continuas en el esquema.

No Funcionales:

- Disponibilidad:** El catálogo de productos y reseñas debe estar disponible 24/7 con latencia <100ms.
- Escalabilidad:** El sistema debe soportar el crecimiento histórico de pedidos sin degradar el rendimiento actual (mitigado vía particionamiento).

- **Mantenibilidad:** El esquema relacional debe alcanzar mínimo la 3FN para evitar redundancias de actualización.

Identificación de Entidades del Dominio

A partir del análisis exploratorio del dataset (Olist), se identifican las siguientes entidades core:

- 1. **Customers (Clientes):** Compradores registrados en la plataforma.
- 2. **Sellers (Vendedores):** Proveedores asociados al marketplace.
- 3. **Products (Productos):** Artículos disponibles para la venta.
- 4. **Orders (Pedidos):** Transacciones maestras de compras realizadas.
- 5. **Order Items (Ítems del pedido):** Detalle de productos adquiridos en cada pedido.
- 6. **Order Payments (Pagos):** Registro financiero de los pedidos.
- 7. **Geolocation (Geolocalización):** Ubicaciones espaciales por código postal.
- 8. **Reviews (Reseñas):** Calificaciones y comentarios dados por los clientes.
- 9. **User Behavior Logs:** Eventos analíticos (vistas, adiciones al carrito).

Tabla de Volúmenes de Datos Esperados

Basados en el EDA previo del dataset Olist, las estimaciones iniciales de volumen son:

Entidad	Origen de Datos	Volumen Esperado (Filas aprox.)	Crecimiento Esperado
Customers	olist_customers_dataset	~99,400	Medio
Sellers	olist_sellers_dataset	~3,100	Bajo
Products	olist_products_dataset	~33,000	Medio
Orders	olist_orders_dataset	~99,400	Alto (Transaccional)
Order Items	olist_order_items_dataset	~112,600	Alto (Transaccional)
Order Payments	olist_order_payments_dataset	~103,800	Alto (Transaccional)
Reviews	olist_order_reviews_dataset	~99,200	Alto (Lectura intensiva)
Geolocation	olist_geolocation_dataset	~1,000,000	Estático / Muy Bajo

C. Diseño Conceptual

1. Diagrama Entidad-Relación (ER)

El modelo conceptual se diseñó para capturar con precisión las interacciones del marketplace, normalizando ubicaciones y conectando transacciones a productos y vendedores.

(El diagrama visual generado vía PlantUML se adjunta o incrusta aquí desde el archivo ER_Diagram_Ecommify.puml).

2. Descripción Detallada de Entidades y Relaciones

Entidad	Descripción Funcional	Relaciones Principales
---------	-----------------------	------------------------

Customers	Almacena los perfiles de los compradores. <code>customer_id</code> es único por cada pedido (clave transaccional), mientras que <code>customer_unique_id</code> rastrea al usuario a través del tiempo.	1-N con <code>Orders</code> (un cliente puede tener muchos pedidos). N-1 con <code>Geolocation</code> .
Sellers	Proveedores de los productos. Contiene la ubicación desde donde se despacha el producto.	1-N con <code>Order_Items</code> . N-1 con <code>Geolocation</code> .
Geolocation	Diccionario geográfico que vincula el prefijo del código postal (<code>zip_code_prefix</code>) con la ciudad, estado y coordenadas espaciales exactas.	1-N con <code>Customers</code> y <code>Sellers</code> .
Products	Catálogo de artículos disponibles, incluyendo sus atributos técnicos, categoría y fotografías.	1-N con <code>Order_Items</code> .
Orders	Cabecera del pedido que almacena fechas clave (compra, aprobación, entrega) y el estado general de la transacción.	1-N con <code>Order_Items</code> , <code>Order_Payments</code> y 1-N con las <code>Reviews</code> (vía NoSQL).
Order Items	Detalle a nivel de línea para cada pedido. Registra el producto específico vendido, el vendedor que lo despacha, el precio y el valor del flete.	N-1 con <code>Orders</code> , <code>Products</code> y <code>Sellers</code> .
Order Payments	Registros financieros de un pedido. Un mismo pedido puede pagarse con una combinación de tarjetas y vouchers (secuencia de pagos).	N-1 con <code>Orders</code> .

3. Restricciones de Negocio Identificadas

Basado en el contexto del dataset público de Olist (Kaggle), se establecen las siguientes reglas operativas que la base de datos debe hacer cumplir:

- Restricción de Pagos Múltiples:** Un único pedido (`order_id`) puede ser dividido en múltiples métodos de pago (ej. pago parcial con voucher y saldo con tarjeta de crédito), por lo cual la tabla `order_payments` requiere una clave primaria compuesta por `order_id` y `payment_sequential`.
- Límites de Puntaje de Reseñas:** El campo `review_score` (aunque se traslade al modelo de MongoDB) tiene un `CONSTRAINT` o regla de validación estricta de que su valor numérico debe estar entre 1 y 5.
- Dependencia Transaccional:** No pueden existir ítems de pedido (`order_items`) ni pagos (`order_payments`) "huérfanos". La clave foránea con borrado o actualización en cascada está restringida; un pedido no puede ser eliminado si tiene transacciones financieras atadas.
- Envíos Multivendedor:** Un solo pedido (`orders`) puede contener múltiples productos de diferentes vendedores (`sellers`). Por ello, el `freight_value` (valor del flete) se calcula y almacena a nivel de `order_item`, no a nivel del pedido principal.
- Historial de Clientes:** En el dataset de Olist, el `customer_id` actúa como una llave única por cada compra (sesión de compra), mientras que el `customer_unique_id` es el verdadero identificador del usuario físico. Las consultas de lealtad o de "compras repetidas" deben hacerse sobre `customer_unique_id`.

D. Diseño Lógico - Módulo PostgreSQL

El modelo relacional ha sido diseñado y normalizado hasta la Tercera Forma Normal (3FN), garantizando la integridad de los datos y minimizando la redundancia. El proceso de normalización se realizó de la siguiente manera:

- **Primera Forma Normal (1FN):** Partiendo del dataset denormalizado original de Olist (donde existían listas separadas por comas o atributos repetitivos de ítems dentro de un mismo pedido), se descompuso la información asegurando que cada columna contenga valores atómicos. Por ejemplo, los ítems de un pedido se separaron en la tabla `order_items` en lugar de estar anidados dentro de la tabla `orders` .
- **Segunda Forma Normal (2FN):** Se identificaron y eliminaron las dependencias parciales en tablas con claves compuestas. En la tabla `order_items` (cuya PK es `order_id` + `order_item_id`), los atributos como `category_name` o `product_weight` dependían únicamente del `product_id` (y no de todo el ítem del pedido), por lo que se extrajeron a su propia tabla maestra `products` .
- **Tercera Forma Normal (3FN):** Se eliminaron las dependencias transitivas asegurando que cada atributo no clave dependa única y directamente de la clave primaria. Por ejemplo, la información de latitud y longitud dependía del código postal (`zip_code_prefix`), no directamente del cliente (`customer_id`) o del vendedor (`seller_id`). Por tanto, se creó la entidad independiente `geolocation` , referenciada mediante clave foránea (FK).

A continuación, se detalla el esquema lógico tabla por tabla resultante de este proceso de normalización.

1. Diccionario Estructural Tabla por Tabla

Tabla: `geolocation`

Columna	Tipo de Dato	Claves / Constraints	Descripción / Justificación
<code>zip_code_prefix</code>	<code>VARCHAR(10)</code>	PK	Código postal, clave primaria.
<code>lat</code>	<code>FLOAT</code>	NOT NULL	Latitud original.
<code>lng</code>	<code>FLOAT</code>	NOT NULL	Longitud original.
<code>geom</code>	<code>GEOMETRY(Point, 4326)</code>	Índice GIST	Tipo Avanzado: Facilita el uso de la extensión PostGIS para cálculos matemáticos espaciales rápidos.
<code>city</code>	<code>VARCHAR(100)</code>		Nombre de la ciudad.
<code>state</code>	<code>VARCHAR(2)</code>		Siglas del estado.

Tabla: `customers`

Columna	Tipo de Dato	Claves / Constraints	Descripción / Justificación
<code>customer_id</code>	<code>VARCHAR(50)</code>	PK	ID transaccional por cada sesión de compra.
<code>customer_unique_id</code>	<code>VARCHAR(50)</code>	NOT NULL	Identificador único y real del cliente.
<code>zip_code_prefix</code>	<code>VARCHAR(10)</code>	FK -> geolocation	Relaciona la ubicación del comprador.

Tabla: **sellers**

Columna	Tipo de Dato	Claves / Constraints	Descripción / Justificación
seller_id	VARCHAR(50)	PK	Identificador del vendedor.
zip_code_prefix	VARCHAR(10)	FK -> geolocation	Ubicación de despacho.

Tabla: **products**

Columna	Tipo de Dato	Claves / Constraints	Descripción / Justificación
product_id	VARCHAR(50)	PK	Identificador único del producto.
category_name	VARCHAR(100)		Categoría principal.
product_name	TEXT	Índice GIN	Indexado con la extensión pg_trgm para búsqueda difusa (fuzzy search) de nombres.
specifications	JSONB	Índice GIN	Tipo Avanzado: Permite almacenar longitud, ancho, altura y peso sin requerir múltiples columnas nulas, ideal para un catálogo donde las especificaciones varían por categoría.
photos	TEXT []		Tipo Avanzado (Array): Evita la creación de una tabla anexa product_photos, optimizando las consultas.

Tabla: **orders (Particionada)**

Columna	Tipo de Dato	Claves / Constraints	Descripción / Justificación
order_id	VARCHAR(50)	PK (compuesta)	Identificador del pedido.
customer_id	VARCHAR(50)	FK -> customers	Cliente que realiza el pedido.
status	VARCHAR(20)	NOT NULL	Estado del pedido (ej. delivered, shipped).
purchase_timestamp	TIMESTAMP	PK (compuesta)	Particionamiento: Clave de partición. Se particiona la tabla por rangos (ej. semestral/anual) para que los pedidos antiguos no ralenticen los índices transaccionales actuales.

Tablas Detalle: **order_items** y **order_payments**

- **order_items** : Clave primaria compuesta por (**order_id** , **purchase_timestamp** , **order_item_id**). Sus foráneas apuntan a **products** y **sellers** . Registra el **price** y el **freight_value** .

- `order_payments` : Clave primaria compuesta por (`order_id` , `purchase_timestamp` , `payment_sequential`). Registra el valor, tipo y cuotas de pago.

2. Justificación de Extensiones

- **PostGIS:** Incorporada para aprovechar las coordenadas de compradores y vendedores, y así habilitar proyecciones de rutas de envío más exactas que simplemente agrupar por ciudad o estado.
- **pg_trgm:** En un e-commerce, los usuarios comenten errores de escritura. Esta extensión habilita la generación de índices trigramáticos sobre `product_name` , permitiendo respuestas rápidas con el operador `LIKE` o `%` sin que decaiga el performance.

E. Diseño Lógico Preliminar - Módulo MongoDB

El catálogo enriquecido y la actividad de los usuarios requieren flexibilidad estructural y alta disponibilidad de lectura que superan el alcance de un motor puramente relacional.

1. Patrones de Modelado NoSQL Utilizados

- **Extended Reference Pattern:** Se utiliza en el catálogo de productos para traer la información más relevante a la aplicación del usuario sin necesidad de realizar múltiples JOINS complejos.
- **Subset Pattern:** Para las reseñas (`reviews`), se decidió embeberlas dentro del documento del producto. Como las reseñas más recientes o útiles son las únicas que se leen en la carga inicial de la página del producto, esto reduce dramáticamente la latencia.

2. Colecciones Identificadas y Esquema de Documentos

Colección: `product_catalog` Esta colección es responsable de alimentar el front-end de la tienda.

```
{
  "_id": "uuid-producto",
  "category": "electronics",
  "title": "Smartphone X",
  "price": 499.99,
  "dynamic_attributes": {
    "weight_g": 500,
    "dimensions": {"length": 10, "height": 2, "width": 5}
  },
  "reviews": [
    {
      "review_id": "uuid-review-1",
      "score": 5,
      "title": "Excelente",
      "comment": "Llegó a tiempo",
      "date_created": "2018-05-12T10:00:00Z"
    }
  ],
  "average_score": 4.5
}
```

Colección: `user_behavior_logs` Colección analítica que recibe alto volumen de escritura pero no bloquea las compras. Ideal para almacenar clics, eventos de "agregar al carrito" y tiempos de sesión.

```
{
  "_id": "uuid-evento",
  "customer_id": "uuid-customer",
  "session_id": "uuid-session",
  "event_type": "add_to_cart",
  "metadata": {
    "product_id": "uuid-producto",
    "time_spent_seconds": 120
  },
  "timestamp": "2018-05-12T10:05:00Z"
}
```

3. Justificación de Reparto (PostgreSQL vs MongoDB)

Se seleccionó PostgreSQL para almacenar las entidades de la compra (Orders, Payments) porque estas transacciones exigen garantías de atomicidad estricta y aislamiento (ACID); no podemos permitir fallas donde un pago ingrese pero el inventario no se descuente. Por otro lado, MongoDB alberga el `product_catalog` y los `reviews` porque el proceso de búsqueda y visualización masiva de un catálogo necesita escalabilidad horizontal rápida, y si el catálogo visual se retrasa unos milisegundos respecto al maestro relacional, el negocio no se ve drásticamente afectado.

F. Decisiones Arquitectónicas Justificadas

1. Análisis del Teorema CAP y Matriz de Decisión

La arquitectura de Ecommify se divide en dos dominios bajo las restricciones del Teorema CAP:

Entidad / Módulo	Motor	Atributo CAP Priorizado	Justificación Técnica
Orders & Payments	PostgreSQL	CP (Consistencia y Tolerancia a Particiones)	Requisito estricto de atomicidad (ACID). Una falla de red debe abortar la compra antes que dejar datos financieros inconsistentes.
Geolocation	PostgreSQL	CP	Uso nativo de PostGIS; requiere integridad referencial con los vendedores y clientes.
Product Catalog	MongoDB	AP (Disponibilidad y Tolerancia a Particiones)	La navegación de la tienda no debe interrumpirse. Si hay latencia de red, es preferible mostrar datos eventualmente consistentes a mostrar un error de conexión al cliente.
Reviews & Logs	MongoDB	AP	Ingesta masiva y lecturas sin afectar el motor financiero.

2. Trade-offs y Estrategias de Mitigación

Ninguna arquitectura es perfecta; elegir un modelo híbrido introduce desafíos (trade-offs) que deben ser mitigados:

- **Trade-off:** Complejidad Operacional (mantener dos motores de BD).
 - *Mitigación:* Uso de plataformas administradas (DBaaS) como Supabase (Postgres) y MongoDB Atlas, reduciendo la carga de administración de infraestructura.
- **Trade-off:** Datos duplicados/desincronizados entre el catálogo (Mongo) y las compras (Postgres).
 - *Mitigación:* Implementación de consistencia eventual.
- **Flujo de Sincronización:** Se propone un flujo basado en *Change Data Capture* (CDC), como Debezium o Kafka. Cuando se inserta un nuevo producto o se actualiza un precio en PostgreSQL, el CDC captura el evento y lo empuja hacia MongoDB para mantener el catálogo actualizado de forma asíncrona.

G. Anexos

1. Diccionario de Datos Consolidado

Se resumen los tipos de datos principales. (Para ver las restricciones completas, referirse a la Sección D).

- `customer_id` (PK), `order_id` (PK), `seller_id` (PK): UUID o cadenas hash únicas.
- `price`, `freight_value`, `payment_value` : Decimales para precisión financiera.
- `geom` : Puntos vectoriales SRID 4326.
- `specifications` : JSONB para atributos dinámicos (ancho, alto, peso).
- `photos` : Array de Textos conteniendo URIs.

2. Scripts SQL Preliminares

El código completo del esquema DDL se encuentra versionado en el archivo adjunto:

`postgresql/schema/01_ddl_tables.sql`

3. Ejemplos de Datos Reales (Data Mockup)

Ejemplo - PostgreSQL (Tabla `order_items`):

order_id	purchase_timestamp	order_item_id	product_id	seller_id	price
00010242fe8...	2017-09-13 08:59:02	1	4244733e06...	48436dade1...	58.90

Ejemplo - MongoDB (Colección `product_catalog`):

```
{
  "_id": "4244733e06e7ecb4970a6e2683c13e61",
  "category": "cool_stuff",
  "title": "Super Cool Gadget",
  "price": 58.90,
  "dynamic_attributes": {
    "weight_g": 650,
    "dimensions": {"length_cm": 28, "height_cm": 9, "width_cm": 14}
  },
  "reviews": [
    {
      "review_id": "97ca439bc427b48bc1cd7177abe71365",
      "score": 5,
      "title": "Muito bom",
      "comment": "Produto entregue antes do prazo."
    }
  ]
}
```

```
    "date_created": "2017-09-20T00:00:00Z"  
  },  
  1,  
  "average_score": 5.0  
}
```